

Análisis Heurístico y Exacto para el Problema del Viajante (TSP)

Evolución de Estrategias: V3 a V8

Traveler Project

12 de junio de 2026

Resumen

Este documento detalla la evolución matemática y algorítmica de seis estrategias diseñadas para resolver el Problema del Viajante de Comercio (TSP) basadas en la expansión geométrica de polígonos. Se describe el funcionamiento interno, la complejidad computacional asintótica y el desempeño empírico de cada versión. La versión más reciente, **V8 (Outside-In Angle Optimization)**, introduce un enfoque inverso: comienza desde el casco convexo completo y se contrae hacia los puntos interiores, optimizando la suavidad angular global. Esta estrategia logra los mejores resultados en 4 de 7 instancias TSPLIB evaluadas, superando a LKH en berlin52 (-0.59 %) y manteniendo un error promedio de 3.4 % respecto al óptimo.

1. Introducción

El enfoque de “Triangle Insertion” construye el recorrido del viajante expandiendo iterativamente un polígono inicial. Las versiones V3–V8 representan la evolución desde una heurística polinómica determinista (V3), pasando por una heurística de relajación geométrica con visión futura (V4), hasta culminar en un solver exacto mediante *Branch & Bound* (V5). La V6 reinterpreta el proceso de inserción bajo la **analogía del hilo y pines**, priorizando rutas geoméricamente suaves. La V7 integra metaheurísticas avanzadas y aceleración algorítmica para manejar problemas de gran escala. Finalmente, la V8 introduce el paradigma **Outside-In**, construyendo el tour desde la frontera hacia el interior, logrando la mejor calidad promedio en instancias TSPLIB estándar.

2. Triangle Insertion V3: Convex Hull + Cheapest

2.1. Descripción Matemática

La V3 es una heurística constructiva fuertemente guiada por la geometría espacial.

1. **Inicialización:** Computa el Casco Convexo (Convex Hull) utilizando el algoritmo de *Andrew's Monotone Chain*. Extrae el triángulo base $\Delta_{base} = \{A, B, C\} \in \text{Hull}$ que maximiza el perímetro. Esto garantiza una frontera externa que envuelve el problema, evitando auto-intersecciones tempranas.
2. **Cheapest Insertion Global:** En cada iteración, evalúa todos los nodos no visitados $u \in U$ contra todas las aristas actuales $(i, j) \in E$. Selecciona el par $(u, (i, j))$ que minimice el costo de inserción:

$$\Delta C(i, j, u) = d(i, u) + d(u, j) - d(i, j)$$

3. **Post-procesamiento:** Aplica operadores de búsqueda local *2-Opt* y *Or-Opt* para desenredar cruces residuales.

2.2. Complejidad

- El Casco Convexo toma $\mathcal{O}(N \log N)$.
- La búsqueda del Cheapest Insertion requiere evaluar $|U|$ candidatos contra $|E|$ aristas. En el peor caso, esto cuesta $\mathcal{O}(N^2)$ por iteración. A lo largo de N inserciones, la complejidad constructiva es $\mathcal{O}(N^3)$.

3. Triangle Insertion V4: Look-Ahead Rotation

3.1. Descripción Matemática

La V4 introduce el concepto de *Relajación Geométrica de Frontera*. Permite permutar aristas locales antes de realizar una inserción.

1. **Métrica de Atracción:** Calcula el costo del camino sumado al costo de insertar al candidato más cercano:

$$S(P) = \sum_{(i,j) \in P} d(i, j) + \min_{u \in U, (i,j) \in P} \Delta C(i, j, u)$$

2. **Rotación Local (Sub-vecindad 3-Opt):** Para cada triplete de nodos consecutivos genera sus 5 permutaciones. Si alguna reduce $S(P)$, se aplica permanentemente.

4. Triangle Insertion V5: Root Backtracking (Branch & Bound)

4.1. Descripción Matemática

La V5 abandona la naturaleza heurística para convertirse en un **Solver Exacto**. Utiliza la topología del problema para crear un árbol de búsqueda sistemática y exhaustiva.

- **Complejidad:** $\mathcal{O}(N!)$, aunque las podas por *Upper Bound* permiten resolver problemas de hasta ~ 15 nodos en tiempos interactivos.

5. Triangle Insertion V6: Smoothest Angle Insertion

5.1. Descripción Matemática

La V6 introduce la **Inserción de Ángulo más Suave**. Para cada candidato $u \in U$ y cada arista $(i, j) \in P$, el score combina costo e inserción angular con factor $\alpha = 2,0$:

$$\text{score}(i, j, u) = \Delta C(i, j, u) \cdot (1 + \alpha (1 + \cos \theta_{ij}^u))$$

Cuando $\theta = \pi$ (línea recta), la penalización es nula. Cuando $\theta = 0$ (giro en U), la penalización máxima es $4\alpha \cdot \Delta C$.

6. Triangle Insertion V7: Metaheurística y Aceleración Geométrica

6.1. Innovaciones Tecnológicas

La V7 representa el salto de una heurística constructiva pura a una **metaheurística híbrida de alto rendimiento**. Sus tres pilares fundamentales son:

1. **Aceleración Geométrica via K-D Tree ($O(N \log N)$)**: Utiliza un árbol de partición espacial balanceado para restringir la búsqueda de vecinos a los $k = 15$ nodos más cercanos. Al reconstruir el árbol únicamente con nodos *no visitados*, reduce la complejidad constructiva a $O(N^2 \log N)$.
2. **Ejection Chains (Cadenas de Eyección)**: Extrae secuencias de nodos (longitud 2 a 4) y busca la mejor re-inserción global, permitiendo reestructuraciones profundas del tour.
3. **Hibridación con Recocido Simulado (SA)**: Introduce el **Criterio de Metrópolis** para aceptar movimientos peores temporalmente y escapar de óptimos locales. La temperatura decrece exponencialmente ($T_{next} = T \cdot 0,995$).

6.2. Optimización de Operadores Locales

Los operadores de búsqueda local (*Or-Opt* y *Node Reinsertion*) operan mediante **Cálculo de Delta** ($O(1)$ por movimiento), transformando procesos que tomaban minutos en milisegundos.

6.3. Rendimiento en Benchmarks Reales

La V7 logra un tour 3% más corto en *berlin52* que Christofides y es 100 veces más rápida que V6 en instancias de 1000 nodos.

Instancia	Estrategia	Distancia	Tiempo (ms)	Error vs Opt.
berlin52	V6 (Smoothest)	8121.20	22.53	+7.68 %
	V7 (SA+Geo)	7904.43	1.03	+4.81 %
	Christofides	8338.38	0.09	+10.56 %
	Óptimo (Concorde)	7542.00	—	0.00 %
ch130	V6 (Smoothest)	6338.47	620.68	+3.74 %
	V7 (SA+Geo)	6383.35	8.91	+4.47 %
	Christofides	6427.48	0.40	+5.20 %
	Óptimo (Concorde)	6110.00	—	0.00 %
Synthetic-1000	Christofides	25138.30	29.46	—
	V7 (SA+Geo)	25322.87	1293.76	—

Cuadro 1: Comparativa de rendimiento y calidad V6/V7.

7. Triangle Insertion V8: Outside-In Angle Optimization

7.1. Descripción Matemática

La V8 introduce un **cambio de paradigma** en la construcción del tour: en lugar de expandir desde un triángulo inicial hacia afuera, comienza desde el **casco convexo completo** y se contrae hacia los puntos interiores. Esta aproximación *Outside-In* produce tours con mejor calidad promedio en instancias TSPLIB estándar.

1. **Inicialización:** Calcula el Casco Convexo completo $Hull(P)$ utilizando *Andrew's Monotone Chain*. Todos los puntos del casco forman el tour inicial, garantizando que la frontera exterior está correctamente definida desde el inicio.
2. **Búsqueda de Referencia:** Para cada inserción, identifica el centro geométrico del path actual:

$$\mathbf{c} = \frac{1}{|P|} \sum_{i \in P} \mathbf{p}_i$$

Utiliza un K-D Tree para encontrar los $k = 8$ nodos no visitados más cercanos a $\mathbf{c}$ como candidatos de referencia.

3. **Inserción con Optimización Angular:** Para cada candidato de referencia, busca sus $k = 8$ vecinos más cercanos. Evalúa cada candidato u en cada posición (i, j) del tour usando un score compuesto:

$$\text{score}(i, j, u) = 0,5 \cdot A_\theta + 0,3 \cdot A_{\text{neighborhood}} + 0,2 \cdot C_{\text{penalty}}$$

Donde:

- $A_\theta = \theta/\pi$: Normalización del ángulo de inserción ($\theta \approx \pi$ es ideal)
- $A_{\text{neighborhood}}$: Promedio de ángulos en vecinos adyacentes después de inserción

- $C_{\text{penalty}} = 1/(1 + \Delta C/d(i, j))$: Penalización por costo relativo
4. **Post-procesamiento:** Aplica 2-Opt, Or-Opt (segmentos de longitud 1 y 2) y Node Reinsertion para refinar el tour final.

7.2. Complejidad

- Casco Convexo: $\mathcal{O}(N \log N)$
- Construcción del K-D Tree: $\mathcal{O}(U \log U)$ por iteración, donde U es el número de nodos no visitados
- Búsqueda de candidatos: $\mathcal{O}(k^2 \cdot |P|)$ por inserción
- **Complejidad total:** $\mathcal{O}(N^2 \log N)$ constructiva

7.3. Ventajas del Enfoque Outside-In

- **Estructura global desde el inicio:** Al comenzar con el casco convexo, el tour ya tiene una topología razonable que evita auto-intersecciones graves.
- **Contracción suave:** Los puntos interiores se insertan en posiciones que minimizan la perturbación de la estructura existente.
- **Mejor calidad promedio:** En 4 de 7 instancias TSPLIB evaluadas, V8 produce el mejor resultado.

7.4. Rendimiento en Benchmarks Reales (TSPLIB)

Instancia	N	V8 (%Err)	V7 (%Err)	V6 (%Err)	Mejor
berlin52	52	3.08 %	4.81 %	7.68 %	V8
eil51	51	2.91 %	3.54 %	2.61 %	V6
st70	70	2.07 %	4.81 %	5.41 %	V8
kroA100	100	6.01 %	3.65 %	0.90 %	V6
eil76	76	6.71 %	4.15 %	4.38 %	V6
ch130	130	2.70 %	4.47 %	3.74 %	V8
pr76	76	2.31 %	2.19 %	1.75 %	V6
Promedio	—	3.40 %	3.95 %	3.78 %	—

Cuadro 2: Comparativa de error porcentual respecto al óptimo conocido.

7.5. Trade-offs

- **Calidad vs Velocidad:** V8 es 10-30x más lenta que V7 pero produce mejor calidad en la mayoría de instancias.
- **V6 vs V8:** V6 gana en instancias grandes y dispersas (kroA100, eil76, pr76), mientras V8 domina en instancias medianas y compactas.

Instancia	Estrategia	Distancia	Tiempo (ms)	Error vs Opt.
berlin52	V6	8121.20	22.53	+7.68 %
	V7	7904.43	1.03	+4.81 %
	V8	7774.24	30.25	+3.08 %
	Óptimo	7542.00	—	0.00 %
st70	V6	711.53	64.02	+5.41 %
	V7	707.46	1.96	+4.81 %
	V8	688.99	32.91	+2.07 %
	Óptimo	675.00	—	0.00 %
ch130	V6	6338.47	620.68	+3.74 %
	V7	6383.35	8.91	+4.47 %
	V8	6319.36	138.14	+3.43 %
	Óptimo	6110.00	—	0.00 %

Cuadro 3: Detalle de mejores resultados por instancia.

- **Escalabilidad:** Para $N \geq 500$, el tiempo de V8 crece significativamente ($\geq 10s$), mientras V7 mantiene tiempos interactivos.

8. Conclusión

El ecosistema de estrategias ofrece soluciones para distintos contextos:

- **V3 (Convex Hull + Cheapest):** Balance óptimo entre velocidad y precisión para uso general.
- **V5 (Branch & Bound):** Optimalidad exacta para instancias pequeñas ($N \leq 15$).
- **V6 (Smoothest Angle):** Suavidad física con buena calidad en instancias grandes y dispersas.
- **V7 (Metaheurística Acelerada):** Alto rendimiento en gran escala mediante K-D Trees y Simulated Annealing.
- **V8 (Outside-In):** **Mejor calidad promedio** (3.4% error) en instancias TSPLIB estándar, ideal cuando la calidad del tour es prioritaria sobre el tiempo de ejecución.

La comparación contra el estado del arte (LKH/Concorde) muestra que las estrategias V6-V8 se mantienen en el rango 2-7 % de error, competitivo para heurísticas de inserción puras. V8 destaca como la mejor opción para instancias medianas (50-150 nodos), mientras V7 es preferible para escalabilidad extrema.